

# Contents

|                                          |          |
|------------------------------------------|----------|
| <b>1 Chapter 4: Loops in PHP</b>         | <b>1</b> |
| 1.1 For Loop                             | 2        |
| 1.1.1 Syntax                             | 2        |
| 1.1.2 Example                            | 2        |
| 1.1.3 When to Use                        | 2        |
| 1.2 Foreach Loop                         | 2        |
| 1.2.1 Syntax                             | 2        |
| 1.2.2 Example                            | 2        |
| 1.2.3 When to Use                        | 3        |
| 1.3 While Loop                           | 3        |
| 1.3.1 Syntax                             | 3        |
| 1.3.2 Example                            | 3        |
| 1.3.3 When to Use                        | 3        |
| 1.4 Do-While Loop                        | 3        |
| 1.4.1 Syntax                             | 4        |
| 1.4.2 Example                            | 4        |
| 1.4.3 When to Use                        | 4        |
| 1.4.4 Conclusion                         | 4        |
| <b>2 Loops alternative syntax</b>        | <b>4</b> |
| 2.1 Alternative Syntax for for Loop      | 4        |
| 2.2 Alternative Syntax for foreach Loop  | 5        |
| 2.3 Alternative Syntax for while Loop    | 5        |
| 2.4 Alternative Syntax for do-while Loop | 5        |
| 2.5 Example in an HTML Template          | 6        |
| 2.6 Conclusion                           | 6        |
| <b>3 Break and Continue in PHP</b>       | <b>6</b> |
| 3.1 The break Statement                  | 6        |
| 3.1.1 Use Case of break:                 | 6        |
| 3.1.2 Example:                           | 6        |
| 3.2 The continue Statement               | 7        |
| 3.2.1 Use Case of continue:              | 7        |
| 3.2.2 Example:                           | 7        |
| 3.3 Note on Using break and continue     | 7        |
| 3.4 Conclusion                           | 7        |

## 1 Chapter 4: Loops in PHP

Loops are a fundamental part of programming in PHP, allowing you to execute a block of code repeatedly under certain conditions. This ability to iterate over data or execute code multiple times is crucial in many programming scenarios. In this chapter, we will cover the four primary types of loops in PHP: `for`, `foreach`, `while`, and `do-while`, along with guidance on when to use each type for optimal results.

## 1.1 For Loop

The `for` loop is ideal when you know in advance how many times you want to execute a statement or a block of statements.

### 1.1.1 Syntax

```
for (initialization; condition; increment) {  
    // Code to be executed  
}
```

### 1.1.2 Example

For example, to print numbers from 1 to 10:

```
for ($i = 0; $i < 10; $i++) {  
    echo $i . " ";  
}  
  
for ($i = 1; $i <= 10; $i++) {  
    echo $i . "\n";  
}
```

### 1.1.3 When to Use

- **Count-Controlled Iteration:** Ideal for looping a specific number of times.
- **Array Indexing:** Useful for iterating over array elements when the index is needed.

## 1.2 Foreach Loop

The `=foreach=` loop is specifically designed for iterating over arrays or objects, making it simpler to work with elements without worrying about indexes or keys.

### 1.2.1 Syntax

```
foreach ($array as $value) {  
    // Code to be executed  
}
```

Or, to access both the key and the value:

```
foreach ($array as $key => $value) {  
    // Code to be executed  
}
```

### 1.2.2 Example

To print all elements of an array>

```
$colors = ["red", "green", "blue"];
```

```
foreach ($colors as $color) {  
    echo $color . " ";  
}
```

To print all values of an associative array:

```
$colors = ["red" => "#FF0000", "green" => "#00FF00", "blue" => "#0000FF"];  
  
foreach ($colors as $color) {  
    echo $color . "\n";  
}
```

### 1.2.3 When to Use

- **Array Iteration:** for iterating through arrays or objects where the order and number of elements are unknown or irrelevant.
- **Readable Code:** Offers clearer and more readable code when dealing with array or objects elements.

## 1.3 While Loop

The `while` loop is used when you want to execute a block of code as long as a specified condition is true.

### 1.3.1 Syntax

```
while (condition) {  
    // Code to be executed  
}
```

### 1.3.2 Example

```
$i = 0;  
while ($i < 10) {  
    echo $i . " ";  
    $i++;  
}
```

### 1.3.3 When to Use

- **Unknown Iterations:** Suitable when the number of iterations is not known in advance.
- **Conditional Control:** Useful when the continuation condition might be affected by processes within the loop.

## 1.4 Do-While Loop

The `do-while` loop is a variant of the `while` loop. This loop will execute the code block once before checking if the condition is true, then it will repeat the loop as long as the condition is true.

### 1.4.1 Syntax

```
do {  
    // Code to be executed  
} while (condition);
```

### 1.4.2 Example

```
$i = 0;  
do {  
    echo $i . " ";  
    $i++;  
} while ($i < 10);
```

### 1.4.3 When to Use

- **Guaranteed Execution:** Ensures that the loop body is executed at least once.
- **Post-Condition Looping:** Ideal when the condition for continuation is determined at the end of each iteration.

### 1.4.4 Conclusion

Understanding the different types of loops and their appropriate use cases is crucial for efficient PHP programming. While `for` loops are best for fixed iterations, `foreach` is optimal for array traversal without needing to know the array size. `while` loops suit scenarios where the number of iterations is not predetermined, and `do-while` loops guarantee at least one iteration, regardless of the condition. As you continue to develop your PHP skills, experiment with these loops to find the best fit for various coding challenges.

## 2 Loops alternative syntax

In PHP, the alternative syntax for loops provides a way to write loops that is often more readable, especially when embedding PHP within HTML. This syntax is available in PHP 8 and earlier versions. The alternative syntax is particularly useful in templating where PHP is interspersed with HTML.

### 2.1 Alternative Syntax for `for` Loop

Instead of using curly braces {}, you use `:` to start and `endfor;` to end the loop.

#### Standard Syntax:

```
for ($i = 0; $i < 10; $i++) {  
    // Code to execute  
}
```

#### Alternative Syntax:

```
for ($i = 0; $i < 10; $i++):  
    // Code to execute  
endfor;
```

## 2.2 Alternative Syntax for foreach Loop

Similar to the for loop, the foreach loop can use endforeach; in place of curly braces.

### Standard Syntax:

```
foreach ($array as $element) {  
    // Code to execute  
}
```

### Alternative Syntax:

```
foreach ($array as $element):  
    // Code to execute  
endforeach;
```

## 2.3 Alternative Syntax for while Loop

The while loop can use endwhile; at the end instead of curly braces.

### Standard Syntax:

```
while ($condition) {  
    // Code to execute  
}
```

### Alternative Syntax:

```
while ($condition):  
    // Code to execute  
endwhile;
```

## 2.4 Alternative Syntax for do-while Loop

The do-while loop can also be written using alternative syntax, though it's less common.

### Standard Syntax:

```
do {  
    // Code to execute  
} while ($condition);
```

### Alternative Syntax:

```
do:  
    // Code to execute  
while ($condition);
```

## 2.5 Example in an HTML Template

Here's how you might use the alternative syntax in an HTML template:

```
<ul>
  <?php foreach ($items as $item): ?>
    <li><?php echo htmlspecialchars($item); ?></li>
  <?php endforeach; ?>
</ul>
```

This example demonstrates how the alternative syntax can make PHP code embedded in HTML more readable.

## 2.6 Conclusion

The alternative syntax for loops in PHP doesn't change the functionality of the loops but offers a way to write code that can be easier to read and maintain, especially in the context of PHP/HTML templates. This syntax is a matter of preference and style, and its usage often depends on the coding standards of a particular project or team.

## 3 Break and Continue in PHP

In PHP, `break` and `continue` are important control structures used within loops (`for`, `foreach`, `while`, `do-while`) to alter the normal flow of the loop. Understanding how to use them correctly can make your loops more efficient and powerful.

### 3.1 The `break` Statement

`break` is used to terminate the execution of a loop prematurely. When `break` is encountered inside a loop, the loop is immediately stopped, and the program control resumes at the next statement following the loop.

#### 3.1.1 Use Case of `break`:

- To exit a loop when a certain condition is met.
- Commonly used in switch-case statements to terminate a case.

#### 3.1.2 Example:

```
for ($i = 0; $i < 10; $i++) {
    if ($i == 5) {
        break; // Exit the loop when $i equals 5
    }
    echo $i . " ";
}
// Output: 0 1 2 3 4
```

## 3.2 The `continue` Statement

`continue` is used to skip the rest of the current loop iteration and proceed with the next iteration. Unlike `break`, `continue` does not terminate the loop but just skips the remaining statements in the current iteration and continues with the next iteration.

### 3.2.1 Use Case of `continue`:

- To skip certain iterations based on a condition.
- Useful for avoiding nested if-else statements inside loops.

### 3.2.2 Example:

```
for ($i = 0; $i < 10; $i++) {  
    if ($i % 2 == 0) {  
        continue; // Skip the rest of the loop for even numbers  
    }  
    echo $i . " ";  
}  
// Output: 1 3 5 7 9
```

## 3.3 Note on Using `break` and `continue`

- `break` and `continue` only affect the innermost loop that encloses them.
- In nested loops, you can specify a numeric argument to indicate how many levels of enclosing loops should be broken out of or continued. For example, `break 2;` will break out of two levels of loops.

## 3.4 Conclusion

`break` and `continue` are powerful tools for controlling the execution flow inside loops. They can make your code more readable and efficient by preventing unnecessary iterations or exiting a loop as soon as a certain condition is met. However, they should be used judiciously to keep the logic of your loops clear and understandable.